

Wintersoldier 프로젝트 재설계 가이드

STM32G431KB · FreeRTOS CMSIS-V2 · CAN 500kbps · BT HC-05 ↔ JDY-31

작성일: 2026-06-30

1. 현재 코드의 문제점

문제	원인	증상
#if (ProjModeState) 범람	단일 소스에서 두 보드 분기	공유 파일에 보드 로직이 섞임
DEBUG_COMM_ENABLE 존재	디버그 플래그가 런타임 동작을 바꿈	릴리즈 시 동작 변경 불확실
초기화 순서 의존성	Comm_Init ↔ Sensor_Init 순서 민감	순서 바뀌면 LDREX 무한루프
HAL_UART_Transmit Lock 충돌	TX 중 Lock 점유 → RX ISR 재등록 실패	BT RX 영구 중단
CAN→BT 전환 시 TX FIFO 소실	HAL_FDCAN_Stop 즉시 FIFO 취소	마지막 0x101 패킷 미전송
ADC DMA IRQ vs LDREX/STREX	DMA IRQ가 UART IT 등록 중 발생	HAL_UART_Receive_IT 무한루프

2. 보드별 주변기기 정리 (IOC 설정 기준)

2-1. 컨트롤러 (Controller)

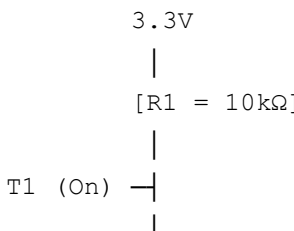
주변기기	용도	설정	DMA	IRQ
FDCAN1	CAN 통신	500 kbps, Classic CAN	X	✓ IT0, Priority 6
USART1	HC-05 BT	115200, 8N1, Async	X	✓ Priority 6
ADC1	굽힘센서 2ch + 3레벨 스위치 1ch	Single Conversion, 폴링, 3채널	X	X
I2C1	ADXL345	Standard Mode 100 kHz, 폴링	X	X
TIM6	HAL Timebase	(SysTick 대신)	X	X
GPIO 입력	CAN/BT 모드 전환 스위치	Pull-Up, 폴링 (EXTI 없음)	—	X
GPIO 출력	HC-05 KEY/EN	Push-Pull, 평시 LOW	—	X

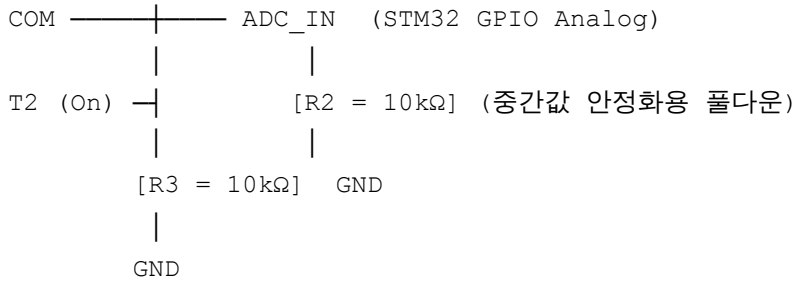
모드 전환은 토글 스위치 → **GPIO 폴링** (Mode_Task 100ms마다 상태 읽기).

EXTI 불필요 — 스위치는 유지 접점이므로 상태가 바뀔 때만 처리하면 됨.

ADC는 3채널 폴링: 굽힘센서 0 · 굽힘센서 1 · 3레벨 스위치.

3레벨 스위치 (On-Com-On) 회로 및 ADC 레벨





스위치 위치	COM 연결	ADC 전압	동작
T1 (Up)	T1 - 3.3V	~3.3V	정방향 이동
Center	미연결	~1.65V	정지
T2 (Dn)	T2 - GND	~0V	역방향 이동

ADC 판정 임계값 (12-bit, 3.3V ref):

```
#define SW3_HIGH_THRESH 3000 /* > 3000 → 정방향 */
#define SW3_LOW_THRESH 500 /* < 500 → 역방향 */
/* SW3_LOW_THRESH ~ SW3_HIGH_THRESH 사이 → 정지 */
```

2-2. 로봇팔 (RobotArm)

주변기기	용도	설정	DMA	IRQ
FDCAN1	CAN 통신	500 kbps, Classic CAN	X	✓ IT0, Priority 6
USART1	JDY-31 BT	115200, 8N1, Async	X	✓ Priority 6
TIM1	서보 0~3 PWM	CH1~CH4, 50 Hz	X	X
TIM2	서보 4~5 PWM	CH1~CH2, 50 Hz	X	X
TIM6	HAL Timebase (SysTick 대신)		X	X

로봇팔에는 모드 전환 입력 없음. 컨트롤러(마스터)의 CommMode를 수신해 자동 팔로우.
ADC, I2C, 버튼 GPIO 모두 불필요.

2-3. 두 보드 NVIC 우선순위

Priority	IRQ	컨트롤러	로봇팔
5	FreeRTOS 기준선	(기준)	(기준)
6	FDCAN1_IT0_IRQn	✓	✓
6	USART1_IRQn	✓	✓
15	SysTick (RTOS 틱)	✓	✓
—	ADC1_2_IRQn	X	—
—	I2C1_EV/ER_IRQn	X	—
—	TIM1_UP / TIM1_CC	—	X
—	TIM2_IRQn	—	X

Priority 6 이상(숫자 큰 것)만 FreeRTOS API 호출 가능.

```
FreeRTOSConfig.h: configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY = 5
```

3. 새 프로젝트 구조

각 프로젝트는 완전히 독립된 CubeIDE 프로젝트다.

공유 폴더 없이 파일을 각자 보유하며, 통신은 양쪽 `comm_protocol.h` 의 패킷 ID·포맷을 동일하게 유지하는 것으로 맞춘다.

3-1. 디렉터리 레이아웃

```
Wintersoldier_Project/      ← git 저장소 루트
|
├─ Controller/             ← CubeIDE 프로젝트 ① (컨트롤러)
|   ├── .project / .cproject (CubeIDE 프로젝트 메타)
|   ├── Controller.ioc      ← 컨트롤러 전용 IOC
|   ├── Drivers/           (CubeIDE 생성 - HAL, CMSIS)
|   ├── Middlewares/       (CubeIDE 생성 - FreeRTOS)
|   └─ Core/
|       ├── Inc/
|       |   ├── main.h      (CubeIDE 생성)
|       |   ├── comm_protocol.h ★ 패킷 ID · 포맷 (RobotArm과 반드시 동일)
|       |   ├── sys_types.h CommMode, BT_ConnState, SysState
|       |   ├── comm_bt_hw.h BT UART HW 계층 API
|       |   ├── comm_can_hw.h CAN HW 계층 API
|       |   ├── sensor_read.h 굽힘센서 · ADXL345 드라이버
|       |   └─ app_ctrl.h   컨트롤러 응용 계층
|       └─ Src/
|           ├── main.c      (CubeIDE 생성 - 주변장치 Init)
|           ├── app_freertos.c (CubeIDE 생성 - 태스크 생성)
|           ├── stm32g4xx_it.c (CubeIDE 생성 - ISR)
|           ├── comm_bt_hw.c BT UART TX/RX, 패킷 파서
|           ├── comm_can_hw.c CAN 필터·TX·RX 콜백
|           ├── sensor_read.c ADC 폴링 + ADXL345 I2C
|           └─ app_ctrl.c   Comm_Task · Mode_Task 구현
|
└─ RobotArm/              ← CubeIDE 프로젝트 ② (로봇팔)
    ├── .project / .cproject
    ├── RobotArm.ioc      ← 로봇팔 전용 IOC
    ├── Drivers/
    ├── Middlewares/
    └─ Core/
        ├── Inc/
        |   ├── main.h      (CubeIDE 생성)
        |   ├── comm_protocol.h ★ 패킷 ID · 포맷 (Controller와 반드시 동일)
        |   ├── sys_types.h
        |   ├── comm_bt_hw.h
        |   ├── comm_can_hw.h
        |   ├── servo_ctrl.h 서보 PWM 드라이버
        |   └─ app_robot.h  로봇팔 응용 계층
        └─ Src/
```

└─ main.c	(CubeIDE 생성)
└─ app_freertos.c	(CubeIDE 생성)
└─ stm32g4xx_it.c	(CubeIDE 생성)
└─ comm_bt_hw.c	BT UART TX/RX, 패킷 파서 (복사본)
└─ comm_can_hw.c	CAN 필터·TX·RX 콜백 (복사본)
└─ servo_ctrl.c	TIM PWM CCR 업데이트
└─ app_robot.c	Comm_Task · Servo_Task · Mode_Task 구현

★ comm_protocol.h 는 양쪽이 항상 동일해야 한다.

패킷 ID, 페이로드 크기, BT SOF/EOF 값이 다르면 통신 불가.

한쪽을 수정하면 다른 쪽에도 즉시 반영할 것.

3-2. CubeIDE 프로젝트 생성 절차

- ① File → New → STM32 Project
MCU: STM32G431KBTx
Project name: Controller
Location: .../Wintersoldier_Project/Controller/
- ② IOC 설정 완료 후 Project → Generate Code
Core/, Drivers/, Middlewares/ 자동 생성
- ③ Core/Inc 와 Core/Src 에 수동 작성 파일 추가
(app_ctrl.c, sensor_read.c, comm_bt_hw.c 등)
- ④ RobotArm/ 도 동일 절차로 별도 생성

IOC에서 "Generate Code" 할 때마다 main.c, app_freertos.c, stm32g4xx_it.c 가 재생성 된다.

직접 작성한 코드는 반드시 /* USER CODE BEGIN */ ... /* USER CODE END */ 안에 넣을 것.

4. 통신 프로토콜

4-1. 패킷 ID 정의 (BT + CAN 공통)

ID	방향	내용	CAN 식별자
PKT_SENSOR (0x01)	Controller → Robot	굽힘센서 2ch · Pitch · Roll	0x100
PKT_CTRL_STATUS (0x02)	Controller → Robot	CommMode · 스위치 · 릴레이	0x101
PKT_ROBOT_STATUS (0x11)	Robot → Controller	모터상태 6ch · LCD 상태	0x202

4-2. BT 패킷 프레임 (13 bytes, UART 115200 bps)

Byte: 0 1 2 3 ~ 10 11 12

SOF	ID	LEN	PAYLOAD	CHECKSUM	EOF
0xA5		0x08	(8 bytes)	XOR	0x5A

CHECKSUM = ID ^ LEN ^ PAYLOAD[0] ^ ... ^ PAYLOAD[7]

4-3. CAN 페이로드 레이아웃 (8 bytes)

PKT_SENSOR (0x100)

Byte:	7	6	5	4	3	2	1	0
	Bend[1]	Bend[0]	Roll	Pitch	uint16	uint16	int16	int16

PKT_CTRL_STATUS (0x101)

Byte 0 비트맵:

- [7:5] reserved
- [4:2] CommMode (0=IDLE, 1=CAN, 2=BT)
- [1] RelayStatus
- [0] SwitchStatus

Byte 1~7: reserved (0x00)

PKT_ROBOT_STATUS (0x202)

Byte 0 비트맵:

- [7] LcdStatus
- [6:1] MotorStatus (bit0=Motor0 ... bit5=Motor5)
- [0] reserved

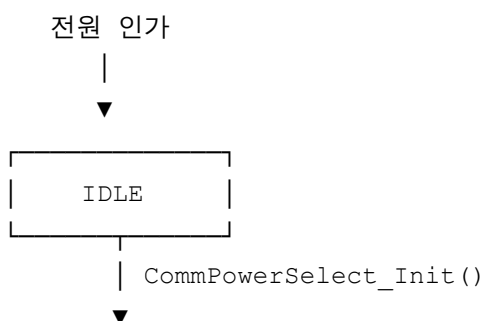
Byte 1:

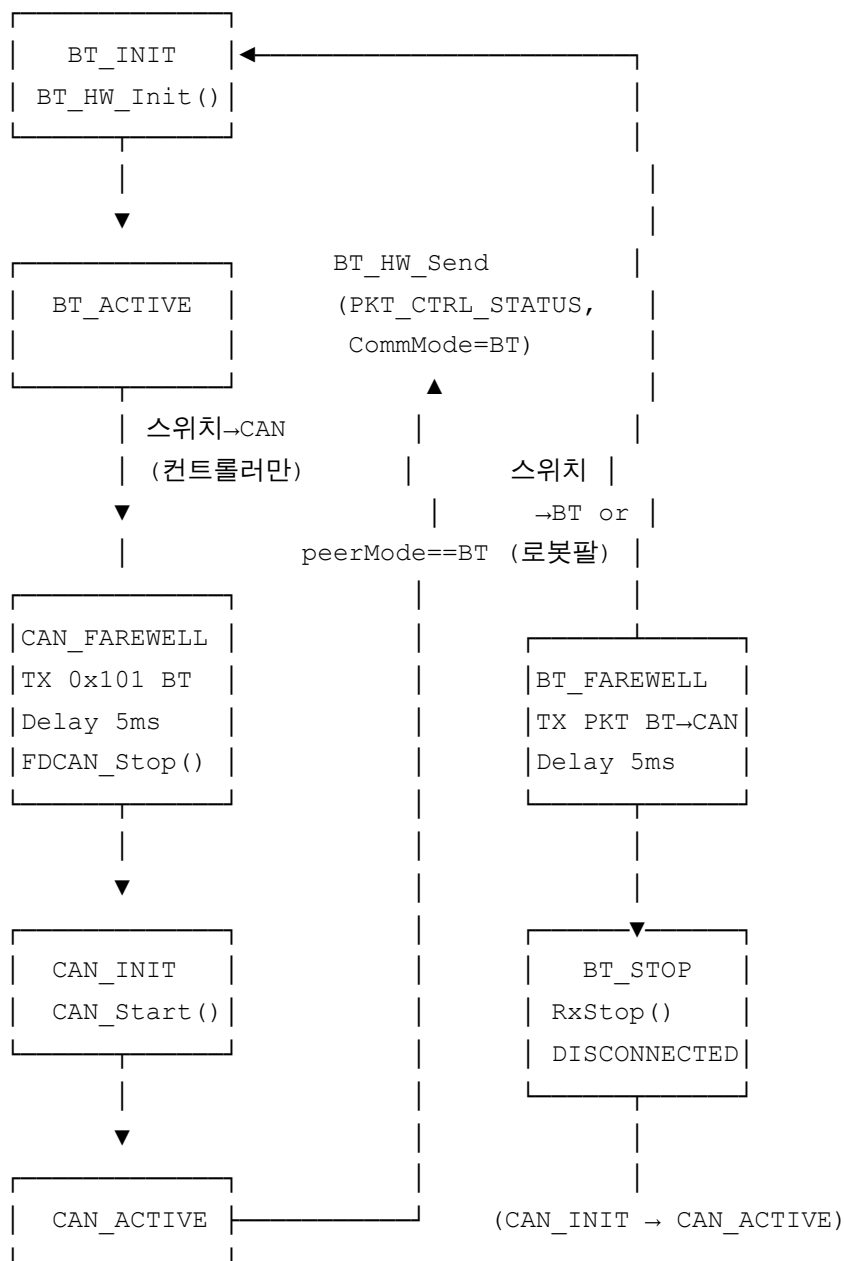
- [4:2] RobotCommMode

Byte 2~7: reserved

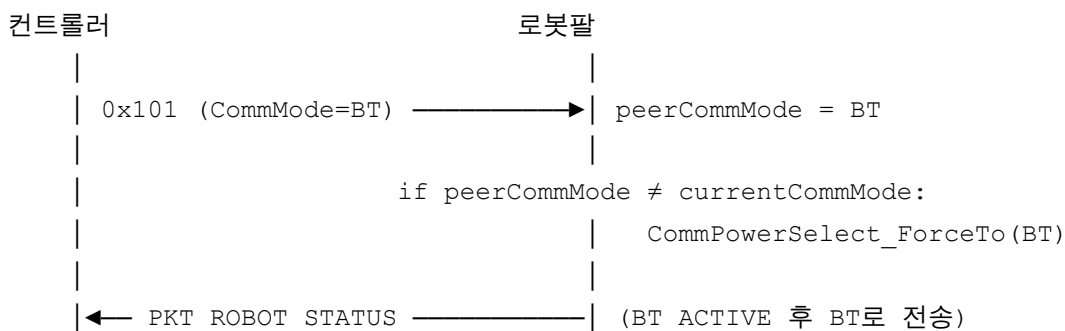
5. CommMode 상태머신

5-1. 전환 다이어그램





5-2. 로봇팔 자동 팔로우



6. FreeRTOS 태스크 설계

설계 원칙: 각 태스크를 **주기별로 분리**하고 `vTaskDelayUntil` 로 정확한 타이밍을 보장한다.
 10ms 태스크(AboveNormal)가 100ms 태스크(Normal)를 선점하므로 별도 Queue/Flag 불필요.

6-1. 컨트롤러 태스크

Controller Board Tasks

Task_10ms [osPriorityAboveNormal] stack=512×4

Sensor_Read()

ADC Ch1: bendingSensor[0]

ADC Ch2: bendingSensor[1]

ADC Ch3: 3-level switch → sw3State

I2C: ADXL345 → gyro_pitch, gyro_roll

Comm_RX_Process()

BT 모드: BT_Receive_Packet() → sysStatus 갱신

CAN 모드: CAN_RX_Queue → sysStatus 갱신

Comm_TX_Sensor()

BT 모드: BT_Pack_And_Send(PKT_SENSOR, remoteSensorTx)

CAN 모드: CAN_Transmit(0x100, remoteSensorTx)

Task_100ms [osPriorityNormal] stack=512×4

Mode_Check()

GPIO_Read(MODE_SW) → 이전값과 비교

변화 감지 시: CommPowerSelect_Apply(newMode)

Comm_TX_Status()

BT 모드: BT_Pack_And_Send(PKT_CTRL_STATUS, status)

CAN 모드: CAN_Transmit(0x101, status)

BT_ConnectionMonitor_100ms() ← 3초 타임아웃 감시

LCD_Update() ← 컨트롤러 전용

Controller 태스크 코드 템플릿

```
/* — Task_10ms (컨트롤러) — */
```

```
void Task_10ms(void const *arg)
```

```
{
```

```
    TickType_t xLastWakeTime = xTaskGetTickCount();
```

```
    for (;;) {
```

```
        Sensor_Read();          /* ADC 3ch + ADXL345 폴링 */
```

```
        Comm_RX_Process();      /* 수신 패킷 파싱 → sysStatus 갱신 */
```

```
        Comm_TX_Sensor();       /* remoteSensorTx 송신 */
```

```
        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(10));
```

```
    }
```

```
}
```

```

/* — Task_100ms (컨트롤러) — */
void Task_100ms(void const *arg)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    for (;;) {
        Mode_Check();                /* GPIO 스위치 폴링 → 모드 전환 */
        Comm_TX_Status();            /* 상태 패킷 송신 */
        BT_ConnectionMonitor_100ms(); /* 3초 무수신 → DISCONNECTED */
        LCD_Update();
        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(100));
    }
}

```

6-2. 로봇팔 태스크



RobotArm 태스크 코드 템플릿

```

/* — Task_10ms (로봇팔) — */
void Task_10ms(void const *arg)
{

```



```

TickType_t xLastWakeTime = xTaskGetTickCount();
for (;;) {
    Comm_RX_Process(); /* 패킷 파싱 → remoteSensorRx / peerCommMode 갱신 */
    Servo_Update(); /* remoteSensorRx → TIM CCR */
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(10));
}
}

/* — Task_100ms (로봇팔) — */
void Task_100ms(void const *arg)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    for (;;) {
        CommMode_AutoFollow(); /* peerCommMode 따라가기 */
        Comm_TX_Status(); /* 로봇팔 상태 송신 */
        BT_ConnectionMonitor_100ms();
        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(100));
    }
}

```

6-3. CubeMX 태스크 등록

CubeMX → FreeRTOS → Tasks and Queues 에 아래와 같이 등록:

Task Name	Priority	Stack Size (Words)	Entry Function
Task_10ms	AboveNormal	512	Task_10ms
Task_100ms	Normal	512	Task_100ms
defaultTask	Normal	128	StartDefaultTask

생성 코드 예시 (main.c USER CODE 영역):

```

osThreadId_t Task10msHandle;
const osThreadAttr_t Task10ms_attributes = {
    .name          = "Task_10ms",
    .stack_size    = 512 * 4,
    .priority      = (osPriority_t) osPriorityAboveNormal,
};

osThreadId_t Task100msHandle;
const osThreadAttr_t Task100ms_attributes = {
    .name          = "Task_100ms",
    .stack_size    = 512 * 4,
    .priority      = (osPriority_t) osPriorityNormal,
};

/* osKernelStart() 전에 */
Task10msHandle = osThreadNew(Task_10ms, NULL, &Task10ms_attributes);
Task100msHandle = osThreadNew(Task_100ms, NULL, &Task100ms_attributes);

```

6-4. 태스크 우선순위 표

태스크	osPriority	FreeRTOS 숫자	주기	스택
TimerDaemon	High	configMAX_PRIORITIES-2	—	256×4
Task_10ms	AboveNormal	3	10ms	512×4
Task_100ms	Normal	2	100ms	512×4
defaultTask	Normal	2	idle	128×4

TimerDaemon 주의: FreeRTOSConfig.h 에서 configTIMER_TASK_PRIORITY = (configMAX_PRIORITIES - 2) 설정.
TimerDaemon이 Task_10ms(AboveNormal=3)보다 높아야 소프트웨어 타이머 콜백이 태스크를 블록하지 않는다.

선점 동작: Task_10ms(AboveNormal)는 Task_100ms(Normal) 실행 중 자동 선점.
vTaskDelayUntil 이 각 태스크의 기준 시각을 누적 관리하므로 선점으로 인한 위상 오차가 없다.

7. IOC 상세 설정

7-1. 클럭 트리 (두 보드 동일)

```
HSI 16MHz → PLL → SYSCLK 170MHz
                |
                └─ AHB 170MHz
                └─ APB1 85MHz (I2C1, USART1, TIM2/6)
                └─ APB2 170MHz (FDCAN1, TIM1, ADC1)
```

HAL Timebase 소스: TIM6 ← SysTick 대신 (FreeRTOS SysTick 충돌 방지)
FDCAN 클럭 소스: PLLQ → 32MHz (IOC: FDCAN Clock Mux = PLL Q clock)

7-2. FDCAN1 (두 보드 동일)

```
Frame Format:           Classic CAN
Auto Retransmission:    Enabled
Tx FIFO Queue Mode:     FIFO mode

Bit Timing (500 kbps, FDCAN_CLK = 32 MHz):
  Prescaler (NBRP): 4 → t_q = 4 / 32 MHz = 125 ns
  TSEG1 (Prop+PS1): 11 t_q
  TSEG2 (PS2): 4 t_q
  SJW: 3 t_q
  Bit time = (1 + 11 + 4) × 125 ns = 2000 ns = 500 kbps ✓

NVIC: FDCAN1_IT0_IRQn Enabled Priority = 6
```

7-3. USART1 (두 보드 동일)

Mode: Asynchronous
Baud Rate: 115200
Word Length: 8 Bits
Stop Bits: 1
Parity: None
Direction: TX + RX
DMA: 사용 안 함 ← RX 인터럽트 방식

NVIC: USART1_IRQn Enabled Priority = 6

TX 방법: 직접 레지스터 huart1.Instance->TDR (HAL_UART_Transmit 금지)
RX 방법: HAL_UART_Receive_IT 1바이트 반복 (HAL_UART_Abort 금지)

7-4. ADC1 (컨트롤러만) — DMA 없음, 3채널 폴링

Conversion Mode: Single Conversion (Continuous 비활성화)

DMA: 사용 안 함

Scan Mode: Disabled

Channels:

Rank 1: IN1 굽힘센서 0 SamplingTime = 12.5 cycles
Rank 2: IN2 굽힘센서 1 SamplingTime = 12.5 cycles
Rank 3: IN3 3레벨 스위치 SamplingTime = 47.5 cycles ← RC 안정화 여유

NVIC: ADC1_2_IRQn Disabled ← 완전히 비활성화

3채널 × 폴링 시간 < 10μs → Comm_Task 10ms 주기에서 충분

ADC 폴링 코드 예시

```
#define SW3_HIGH_THRESH  3000  /* > 3000 → 정방향 */
#define SW3_LOW_THRESH   500   /* < 500  → 역방향  */

static ADC_ChannelConfTypeDef s_ch = {
    .Rank          = ADC_REGULAR_RANK_1,
    .SingleDiff    = ADC_SINGLE_ENDED,
    .OffsetNumber  = ADC_OFFSET_NONE,
};

static uint16_t adc_read(uint32_t ch, uint32_t sampling)
{
    s_ch.Channel    = ch;
    s_ch.SamplingTime = sampling;
    HAL_ADC_ConfigChannel(&hadc1, &s_ch);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 1);
    uint16_t v = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return v;
}
```

```

}

typedef enum { SW3_FORWARD = 1, SW3_STOP = 0, SW3_REVERSE = -1 } Sw3State_t;

static Sw3State_t read_sw3(uint16_t raw)
{
    if (raw > SW3_HIGH_THRESH) return SW3_FORWARD;
    if (raw < SW3_LOW_THRESH) return SW3_REVERSE;
    return SW3_STOP;
}

void Sensor_Read(void) /* Comm_Task 10ms마다 */
{
    remoteSensorTx.bendingSensor[0] = adc_read(ADC_CHANNEL_1, ADC_SAMPLETIME_12CYCLES_5);
    remoteSensorTx.bendingSensor[1] = adc_read(ADC_CHANNEL_2, ADC_SAMPLETIME_12CYCLES_5);
    remoteSensorTx.sw3State = read_sw3(
        adc_read(ADC_CHANNEL_3, ADC_SAMPLETIME_47CYCLES_5))
    ADXL345_Read(&remoteSensorTx.gyro_pitch, &remoteSensorTx.gyro_roll);
}

```

7-5. I2C1 (컨트롤러 ADXL345)

Speed Mode: Standard Mode 100 kHz
 DMA: 사용 안 함 ← HAL_I2C_Master_Receive 블로킹 풀링

NVIC: I2C1_EV_IRQn Disabled
 I2C1_ER_IRQn Disabled

7-6. TIM1 / TIM2 (로봇팔 서보 PWM)

TIM1: CH1~CH4 서보 0~3 (APB2 클럭 170MHz)
 TIM2: CH1~CH2 서보 4~5 (APB1 클럭 85MHz × 2 = 170MHz)

50 Hz PWM 설정 (둘 다 동일):
 PSC = 169 → 타이머 클럭 = 170MHz / 170 = 1MHz
 ARR = 19999 → 주기 = 20ms (50Hz)
 CCR 범위:
 0° → CCR = 1000 (1.0ms)
 90° → CCR = 1500 (1.5ms)
 180° → CCR = 2000 (2.0ms)

NVIC: TIM1_UP, TIM1_CC, TIM2 → 모두 Disabled (CCR 직접 쓰기)

8. 초기화 순서 (main.c USER CODE BEGIN 2)

두 보드 공통 순서:

- | | |
|--|--|
| 1. MX*_Init() | CubeIDE 생성 HAL 초기화 |
| 2. CAN_HW_SetRxCallback(Board_Handler) | 보드별 RX 핸들러 등록 |
| 3. CommPowerSelect_Init() | BT_HW_Init() 포함
↳ HAL_UART_Receive_IT 내부 호출 |
| 4. Sensor_Init() | ADC 캘리브레이션 + ADXL345 초기화
(컨트롤러만. 로봇팔은 생략) |
| 5. osKernelStart() | |

DMA IRQ 별도 비활성화 불필요 - IOC에서 처음부터 사용 안 함으로 설정.

9. CommMode 전환 구현 규칙

9-0. CAN 에러 누적 원칙

상황	TEC/REC 변화	설명
CAN 버스 idle (무트래픽)	변화 없음	RX 전용 노드는 에러 카운터 증가 없음
TX 전송 후 ACK 없음	TEC +8	상대방이 없거나 꺼져 있을 때
노이즈 수신	REC +1	버스에 잡음이 있을 때
TEC ≥ 128	Error Passive	송신 능력 저하
TEC ≥ 256	Bus-Off	버스 완전 분리 → 복구 필요

결론: BT 모드 중 CAN 버스가 idle 이라면 RX 노드의 에러 카운터는 증가하지 않는다.
단, TX 실패(상대방 꺼진 상태에서 전송 시도) 누적 시 Bus-Off 위험이 있으므로
TX는 반드시 활성 모드에서만 하고, Task_100ms에서 Bus-Off 감시가 필요하다.

9-1. 방식 A — 명시적 채널 전환 (이별 패킷 방식)

한 채널을 완전히 닫고 다른 채널을 여는 방식. 에러 누적 위험이 없는 대신 타이밍 관리 필요.

```
/* — CAN → BT 전환 ————— */
/* 마지막 CAN 프레임: 로봇팔에 BT 전환 통보 */
CAN_Transmit(0x101, build_ctrl_status(COMM_MODE_BT));
HAL_Delay(5); /* CAN TX 완료 여유 (ACK 포함 ~1ms) */
HAL_FDCAN_Stop(&hfdcan1); /* FDCAN 완전 중단 → 에러 누적 차단 */
BT_Init(); /* UART RX IT 등록 */
currentCommMode = COMM_MODE_BT;
/* 로봇팔: 0x101 (CommMode=BT) 수신 → Task_100ms AutoFollow → BT 전환 */

/* — BT → CAN 전환 ————— */
/* 마지막 BT 패킷: 로봇팔에 CAN 전환 통보 */
if (g_bt_conn_state == BT_STATE_CONNECTED) {
    BT_Pack_And_Send(PKT_CTRL_STATUS, build_ctrl_status(COMM_MODE_CAN));
    HAL_Delay(5); /* BT TX 완료 여유 (115200bps, 13byte ≈ 1.1ms) */
}
BT_HW_RxStop(); /* UART RX IT 중단 */
g_bt_conn_state = BT_STATE_DISCONNECTED;
hfdcan1.ErrorCode = 0; /* HAL 에러 플래그 클리어 (필수) */
```

```
CAN_Start();
currentCommMode = COMM_MODE_CAN;
/* 로봇팔: PKT_CTRL_STATUS(CommMode=CAN) 수신 → Task_100ms AutoFollow → CAN 전환 */
```

장점: 비활성 채널은 완전히 꺼져 있어 에러 누적 없음

단점: 이별 패킷을 놓치면 로봇팔이 못 따라옴 (BT 연결 불안정 시 취약)

9-2. 방식 B — 암묵적 모드 검출 (항상 양쪽 활성화)

FDCAN과 UART RX IT를 항상 동시에 켜놓고, 유효한 데이터가 오는 쪽으로 모드를 자동 갱신.

```
/* 초기화 (CommPowerSelect_Init) */
hfdcan1.ErrorCode = 0;
CAN_Start(); /* FDCAN 필터 + RX 인터럽트 시작 */
BT_Init(); /* UART RX IT 시작 */
currentCommMode = COMM_MODE_IDLE;

/* CAN RX 콜백 - 체크섬 없이 ID 필터(0x100~0x2FF)로 유효성 판단 */
void HAL_FDCAN_RxFifo0Callback(...) {
    currentCommMode = COMM_MODE_CAN; /* 유효 CAN 프레임 수신 → 모드 갱신 */
    /* 데이터 파싱 ... */
}

/* BT RX 콜백 - 체크섬 통과한 패킷만 모드 갱신 트리거 */
case PARSE_EOF:
    if (byte == BT_EOF && calc_checksum(&g_rx_pkt) == g_rx_pkt.checksum) {
        currentCommMode = COMM_MODE_BT; /* 유효 BT 패킷 수신 → 모드 갱신 */
        g_pkt_ready = 1;
    }
}
```

CAN Bus-Off 복구 (Task_100ms 안에서 주기적 감시):

```
void CAN_ErrorMonitor(void)
{
    /* PSR: Protocol Status Register, BO bit = Bus-Off */
    if (hfdcan1.Instance->PSR & FDCAN_PSR_BO) {
        HAL_FDCAN_Stop(&hfdcan1);
        hfdcan1.ErrorCode = 0;
        hfdcan1.Instance->CCCR &= ~FDCAN_CCCR_INIT; /* Init 모드 해제 */
        CAN_Start(); /* 재시작 - 128x11 recessive bit 후 자동 복구 */
    }
}
```

장점: 이별 패킷 없이 로봇팔이 수신 채널 기준으로 자동 전환

단점: TX를 잘못된 모드에서 보내면 CAN 에러 누적 가능 → Bus-Off 복구 로직 필수

9-3. TX 분기 공통 패턴

어느 방식을 택하든 TX 분기는 동일:

```
void Comm_TX_Sensor(void)
{
    if (currentCommMode == COMM_MODE_CAN) {
        CAN_Transmit(0x100, pack_sensor());
    } else if (currentCommMode == COMM_MODE_BT) {
        BT_Pack_And_Send(PKT_SENSOR, pack_sensor(), SENSOR_LEN);
    }
}
```

9-4. 절대 금지

```
// ✗ HAL_UART_Transmit() → huart Lock 점유 → RX ISR 재등록 실패 → RX 영구 중단
// ✗ HAL_UART_Abort() → HardFault
// ✗ HAL_FDCAN_Start() 전 hfdcan1.ErrorCode 미초기화 → HAL이 재시작 거부
// ✗ Bus-Off 상태에서 TX 계속 시도 → 에러 카운터 무한 증가
```

10. 디버그 방식

빌드 구성을 나누지 않는다. **STM32CubeIDE Debug 빌드 하나로** 진행하고,
volatile 전역 변수를 Live Expression / Memory View 에서 직접 써서 런타임 모드 전환.

10-1. 디버그 전환 변수

각 프로젝트 main.c (또는 comm_power_select.c) 상단에 선언:

```
/* Live Expression 창에서 1(CAN) / 2(BT) 로 쓰면 즉시 전환 */
volatile uint8_t g_dbg_force_mode = 0; /* 0 = 개입 없음 */
```

Task_100ms 안에 한 줄 추가:

```
void Task_100ms(void const *arg)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    for (;;) {
        /* 디버그 강제 전환 */
        if (g_dbg_force_mode == 1 && currentCommMode != COMM_MODE_CAN)
            CommPowerSelect_ForceTo(COMM_MODE_CAN);
        else if (g_dbg_force_mode == 2 && currentCommMode != COMM_MODE_BT)
            CommPowerSelect_ForceTo(COMM_MODE_BT);

        Mode_Check();
        Comm_TX_Status();
        BT_ConnectionMonitor_100ms();
        LCD_Update(); /* 컨트롤러만 */
        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(100));
    }
}
```

```
}  
}
```

10-2. 사용법

목표 디버거에서 할 일

BT 모드로 강제 전환 `g_dbg_force_mode = 2`

CAN 모드로 강제 전환 `g_dbg_force_mode = 1`

스위치 제어로 복귀 `g_dbg_force_mode = 0`

- Live Expression: `g_dbg_force_mode` 추가 → Expressions 창에서 값 수정
- 또는 Memory Browser에서 주소 직접 수정

완성 후 `g_dbg_force_mode` 선언과 `if` 블록 두 줄만 지우면 프로덕션 코드와 동일.

11. HC-05 KEY/EN 핀 권장 처리

현재 KEY/EN 핀 미연결 상태에서 HC-05가 AT 모드로 진입할 수 있다.

(일부 클론 보드: KEY 핀 내부 풀업 → 항상 AT 모드 → LED 고정 ON → JDY-31 미연결)

권장: STM32 GPIO 하나를 HC-05 KEY/EN 핀에 연결

→ 평시: GPIO LOW (데이터 모드)

→ AT 커맨드 필요 시: GPIO HIGH (AT 모드, 38400 baud)

최소 픽스: KEY/EN 핀을 1kΩ 저항으로 GND에 연결 (하드웨어 고정)

12. 마이그레이션 체크리스트

- [] Controller.ioc: ADC 3ch Single Conversion (굽힘x2 + 3레벨스위치), I2C1 폴링, TIM6 Timebase, FDCAN 500kbps, 모드 스위치 GPIO 입력
- [] RobotArm.ioc: TIM1/TIM2 PWM 50Hz, ADC 없음, I2C 없음, 버튼 없음, TIM6 Timebase, FDCAN 500kbps
- [] 두 IOC 모두 NVIC: ADC/I2C/TIM/EXTI IRQ 비활성화, FDCAN IT0 + USART1 Priority=6
- [] `comm_bt_hw.c` TX: 직접 레지스터 (`huart.Instance->TDR`), RX: `HAL_UART_Receive_IT` 1바이트
- [] `comm_can_hw.c` 콜백 패턴 → `#if (ProjModeState)` 완전 제거
- [] 초기화 순서: `CommPowerSelect_Init` → `Sensor_Init` → `osKernelStart`
- [] `configTIMER_TASK_PRIORITY` = `Comm_Task`보다 1 높게 설정
- [] 두 프로젝트 각각 `Debug_BT` / `Debug_CAN` / `Release` 빌드 구성 생성
- [] HC-05 KEY/EN 핀 → GND 연결 (또는 GPIO 제어) 확인